

Project: Build an E-Commerce Data Platform on Databricks

Project Overview

ShopSphere is an e-commerce company that sells products through its website and mobile application. The organization currently stores data across multiple systems.

The transactional application stores operational data in SQL Server,

while product, inventory and customer-behavior data are delivered through files into ADLS Gen2.

The analytics team wants a centralized Lakehouse platform that can integrate these sources and provide trusted, governed and business-ready data.

The final solution should demonstrate:

Source Systems → Ingestion → Incremental Processing → Bronze → Silver → Gold → Governance → Orchestration → Analytics

Business Problem

ShopSphere currently faces several challenges:

Data is distributed across multiple systems.

Operational and analytical workloads are mixed.

Data arrives using different ingestion mechanisms.

New and updated records must be processed incrementally.

Source data contains quality issues.

Business users do not have a single trusted view of sales and customers.

Sensitive customer information needs to be governed.

There is limited visibility into data lineage.

Analytical datasets must be refreshed automatically.

The company wants to answer questions such as:

Sales

What is our daily and monthly revenue?

What is the average order value?

Which products generate the most revenue?

Which categories perform best?

What percentage of orders are cancelled or returned?

Customers

Who are our highest-value customers?

Which customers are active or inactive?

Which customers are at risk of churn?

What categories do customers prefer?

Products

Which products are best sellers?

Which products generate high revenue but low sales volume?

Which products have high views but low purchases?

Customer Behavior

How many customers view products?

How many add products to cart?

What percentage reach checkout?

What percentage actually purchase?

Which products have high cart abandonment?

Inventory

Which products are out of stock?

Which high-selling products have low inventory?

Which products require replenishment?

Project Objective

Build a production-style E-Commerce Lakehouse that:

Ingests transactional data from SQL Server.

Ingests file/event data from ADLS Gen2.

Uses appropriate incremental ingestion strategies.

Stores raw data in the Bronze layer.

Cleans and validates data in Silver.

Integrates data from multiple sources.

Applies meaningful e-commerce business logic.

Creates analytical Gold datasets.

Implements data-quality rules.

Uses Lakeflow Declarative Pipelines / DLT for transformations.

Uses Lakeflow Jobs for orchestration.

Uses Unity Catalog for governance and security.

Provides lineage from source to business output.

Produces business-ready datasets for analytics.

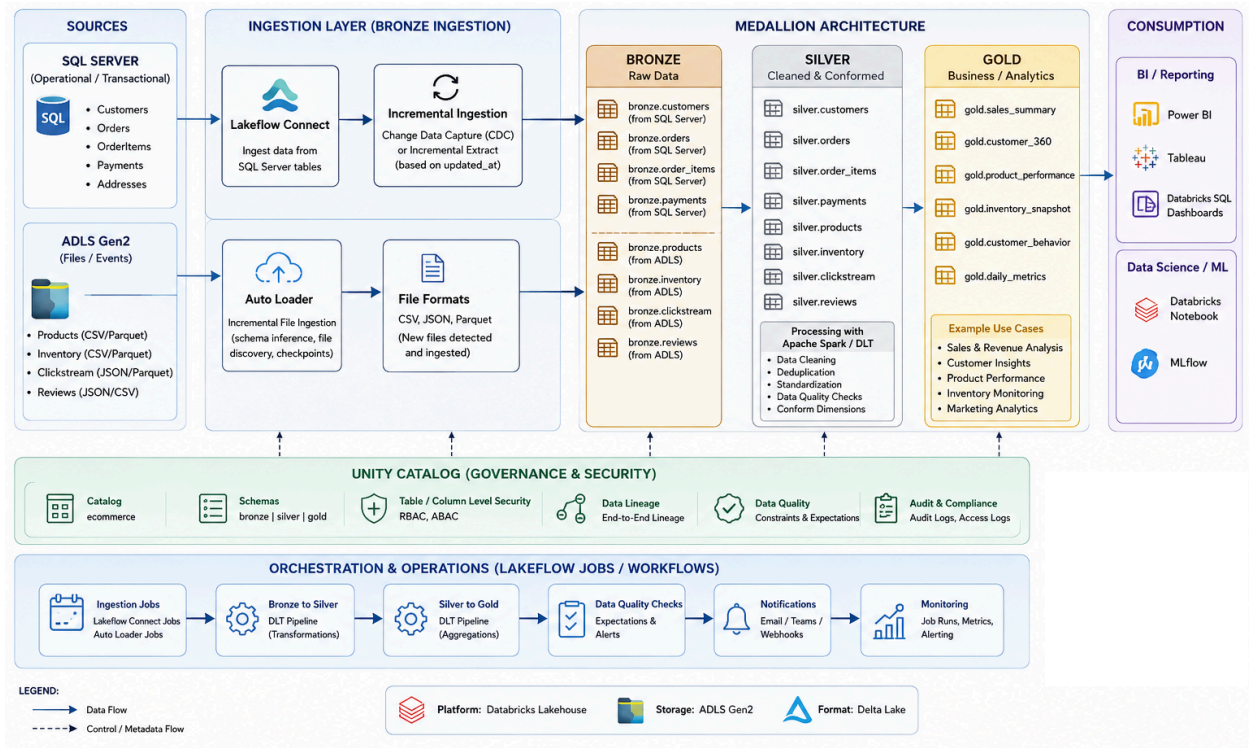
Learning Objectives

By completing the project, participants should be able to:

Concept	Where participants use it
Apache Spark	Transform, join, filter and aggregate data
Lakeflow Connect	Ingest from SQL Server and ADLS
Incremental Ingestion	New/changed records from SQL Server + new files in ADLS

Medallion Architecture	Bronze → Silver → Gold
Unity Catalog	Catalogs, schemas, permissions, lineage
Lakeflow Jobs	Orchestrate the complete pipeline

Project Architecture



Apache Spark

Read and explore structured data.

Work with DataFrames.

Perform filtering, projection and transformations.

Perform joins.

Perform aggregations.

Use window functions.

Handle duplicate and invalid records.

Process large datasets using Spark.

Lakeflow Connect

Connect Databricks to an enterprise database.

Ingest SQL Server data.

Understand source-to-Bronze ingestion.

Incremental Data Ingestion

Understand initial versus incremental loads.

Process new records.

Process changed records.

Ingest newly arrived files.

Understand checkpoint/state concepts.

Avoid unnecessary full reloads.

Medallion Architecture

Design Bronze, Silver and Gold layers.

Keep Bronze source-oriented.

Build trusted Silver datasets.

Create business-oriented Gold datasets.

Data Quality

Identify data-quality problems.

Define expectations.

Handle invalid records.

Validate referential integrity.

Monitor quality metrics.

Unity Catalog

Create catalogs and schemas.

Manage table permissions.

Protect sensitive information.

Understand lineage.

Apply role-based access.

Orchestration

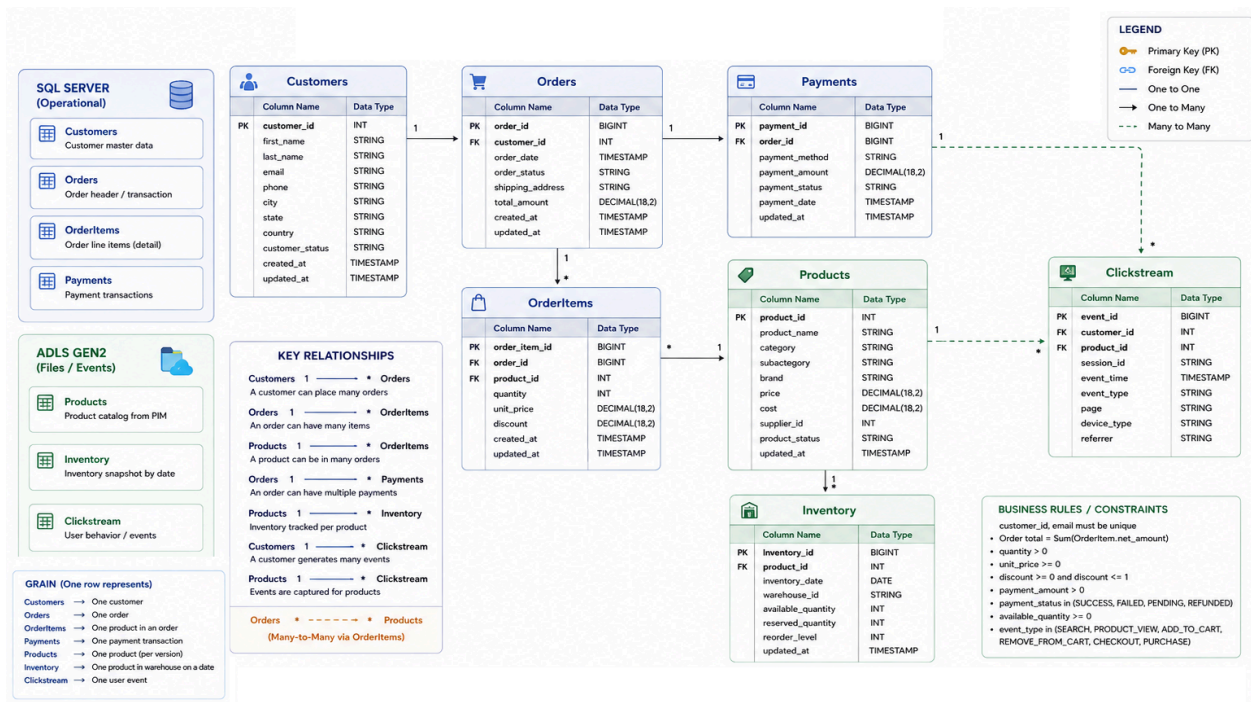
Build multi-step workflows.

Configure dependencies.

Handle failures and retries.

Monitor pipeline execution.

Source System Design



Why SQL Server?

These datasets represent transactions and master data typically maintained by an OLTP application.

Why ADLS?

These datasets represent data that may arrive periodically or continuously as files/events from other systems.

Complete Data Model

Customers

Column	Type	Description
<u>customer_id</u>	INT	Primary key
<u>first_name</u>	STRING	Customer first name
<u>last_name</u>	STRING	Customer last name
email	STRING	Customer email
phone	STRING	Customer phone
city	STRING	City
state	STRING	State
country	STRING	Country
<u>customer_status</u>	STRING	ACTIVE / INACTIVE
<u>created_at</u>	TIMESTAMP	Creation timestamp
<u>updated_at</u>	TIMESTAMP	Last update timestamp

Orders

Column	Type	Description
<u>order_id</u>	BIGINT	Primary key
<u>customer_id</u>	INT	Foreign key → Customers
<u>order_date</u>	TIMESTAMP	Order timestamp
<u>order_status</u>	STRING	Order status
<u>shipping_address</u>	STRING	Shipping address
<u>total_amount</u>	DECIMAL	Order amount
<u>created_at</u>	TIMESTAMP	Creation timestamp
<u>updated_at</u>	TIMESTAMP	Last update

OrderItems

Grain

One row per product within an order.

Column	Type	Description
<u>order_item_id</u>	BIGINT	Primary key
<u>order_id</u>	BIGINT	FK → Orders
<u>product_id</u>	INT	FK → Products
quantity	INT	Quantity purchased
<u>unit_price</u>	DECIMAL	Selling price
discount	DECIMAL	Discount percentage
<u>created_at</u>	TIMESTAMP	Creation timestamp
<u>updated_at</u>	TIMESTAMP	Last update

Payments

Grain

One row per payment transaction.

Column	Type	Description
<u>payment_id</u>	BIGINT	Primary key
<u>order_id</u>	BIGINT	FK → Orders
<u>payment_method</u>	STRING	CARD / UPI / COD / etc.
<u>payment_amount</u>	DECIMAL	Payment amount
<u>payment_status</u>	STRING	SUCCESS / FAILED / etc.
<u>payment_date</u>	TIMESTAMP	Payment timestamp
<u>updated_at</u>	TIMESTAMP	Last update

Products

Grain

One row per product/version in the source feed.

Column	Type	Description
<u>product_id</u>	INT	Primary key
<u>product_name</u>	STRING	Product name
category	STRING	Product category
subcategory	STRING	Product subcategory
brand	STRING	Brand
price	DECIMAL	Selling price
cost	DECIMAL	Product cost
<u>supplier_id</u>	STRING	Supplier
<u>product_status</u>	STRING	ACTIVE / DISCONTINUED
<u>updated_at</u>	TIMESTAMP	Last update

Inventory

Grain

One row per product, warehouse and day.

Column	Type	Description
<u>inventory_date</u>	DATE	Snapshot date
<u>product_id</u>	INT	FK → Products
<u>warehouse_id</u>	STRING	Warehouse
<u>available_quantity</u>	INT	Available stock
<u>reserved_quantity</u>	INT	Reserved stock
<u>reorder_level</u>	INT	Reorder threshold

Clickstream

Grain

One row per customer/system event.

Column	Type	Description
<u>event_id</u>	BIGINT	Primary key
<u>customer_id</u>	INT	Customer, nullable
<u>session_id</u>	STRING	User session
<u>event_time</u>	TIMESTAMP	Event timestamp
<u>event_type</u>	STRING	Event type
<u>product_id</u>	INT	Product, nullable
page	STRING	Page
<u>device_type</u>	STRING	Device

Possible events:

SEARCH

PRODUCT_VIEW

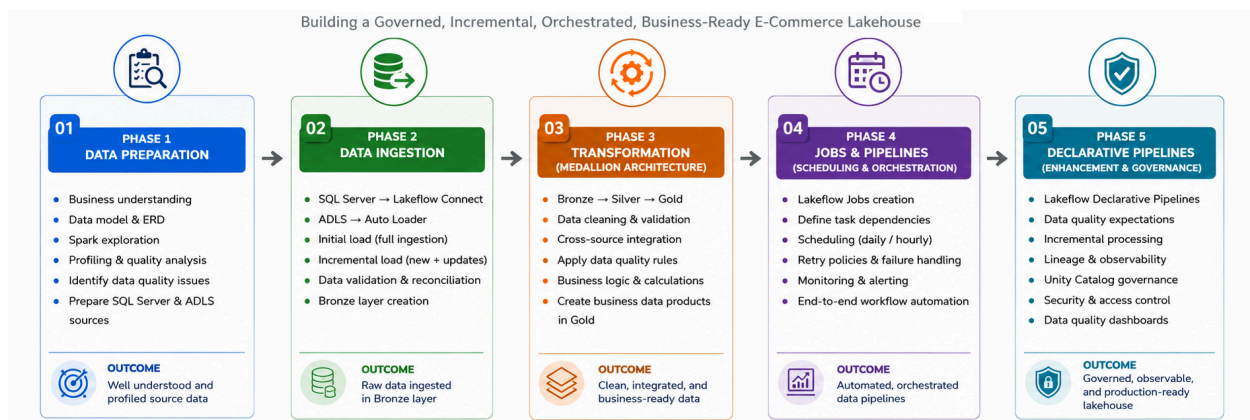
ADD_TO_CART

REMOVE_FROM_CART

CHECKOUT

PURCHASE

Project Implementation



Ingestion Phase

Transactional System Ingestion Phase

Objective

Bring operational data from SQL Server into Databricks.

SQL Server

|

|— Customers

|— Orders

|— OrderItems

|— Payments

Technology

Use: Lakeflow Connect

ADLS Incremental Ingestion

Objective : Implement incremental file ingestion.

Source

ADLS Gen2

|

|— products/

|— inventory/

|— clickstream/

Technology : Auto Loader

Incremental SQL Server Processing

Objective : Demonstrate how new and changed transactional data is handled.

Inserts

customers_new.csv

orders_new.csv

order_items_new.csv

payments_new.csv

Updates

customers_updates.csv

orders_updates.csv

payments_updates.csv

Participants must determine an appropriate strategy for handling:

New records

Updated records

Duplicate records

Phase 02 - Bronze Phase

Bronze Layer

Create the source-aligned raw layer.

Target:

ecommerce.bronze

Tables:

customers

orders

order_items

payments

products

inventory

clickstream

The Bronze layer should preserve source information with minimal transformation. This aligns with the original capstone's requirement that Bronze retain raw data and the original schema in Delta format.

Rules

Participants should avoid:

- Business aggregations

- Complex joins

- Business segmentation

Revenue calculations

Phase 03 - Silver Layer

Objective : Create trusted, standardized and integrated data.

Target:

ecommerce.silver

Tables:

customers

orders

order_items

payments

products

inventory

clickstream

Customer Business Rules

Participants must:

Remove duplicates.

Keep the latest record where appropriate.

Trim whitespace.

Convert email to lowercase.

Validate email format.

Handle missing email/phone.

Standardize city/state.

Validate customer status.

Example:

" JOHN@EXAMPLE.COM "

becomes:

"john@example.com"

Order Business Rules

Participants must:

Remove duplicate orders.

Validate customer IDs.

Standardize status.

Validate order dates.

Reject or quarantine negative order amounts.

Identify cancelled orders.

Identify returned orders.

Identify completed orders.

Standard statuses:

PLACED

CONFIRMED

SHIPPED

DELIVERED

CANCELLED

RETURNED

OrderItem Business Rules

Calculate:

$\text{gross_amount} = \text{quantity} \times \text{unit_price}$

Then:

$\text{discount_amount} = \text{gross_amount} \times \text{discount}$

Then:

$\text{net_amount} = \text{gross_amount} - \text{discount_amount}$

Validation:

quantity > 0

unit_price >= 0

discount >= 0

discount <= 1

This is the first major business transformation.

Silver — Payment Business Rules

Participants must:

Remove duplicates.

Validate payment amount.

Standardize payment method.

Standardize payment status.

Identify failed payments.

Identify successful payments.

Identify refunds.

Valid payment statuses:

SUCCESS

FAILED

PENDING

REFUNDED

Silver — Product Rules

Participants must:

Remove duplicate products.

Standardize product names.

Validate price.

Validate cost.

Validate category.

Standardize product status.

Retain latest product information.

Inventory Rules

Calculate:

`available_to_sell =`

`available_quantity - reserved_quantity`

Then classify:

available_to_sell <= 0

→ OUT_OF_STOCK

available_to_sell <= reorder_level

→ LOW_STOCK

otherwise

→ IN_STOCK

Clickstream Rules

Participants must:

Remove duplicate events.

Validate timestamps.

Standardize event names.

Validate product IDs where present.

Validate customer IDs where present.

Derive event date.

Derive event hour.

Identify anonymous sessions.

Phase 8 — Cross-Source Integration

This phase demonstrates the real value of the Lakehouse.

Participants must combine:

SQL Server

Customers

Orders

OrderItems

Payments

+

ADLS

Products

Inventory

Clickstream

The integration should happen in the Silver/Gold processing layer rather than directly against the operational sources.

Gold Layer

Gold Layer & Business Logic

The Gold layer contains business-ready datasets.

Target:

ecommerce.gold

Minimum required tables:

daily_sales

customer_360

product_performance

customer_behavior

product_conversion

inventory_sales

25. Gold — Daily Sales

Table

gold.daily_sales

Required metrics

sales_date

total_orders

completed_orders

cancelled_orders

returned_orders

gross_sales

discount_amount

net_sales

average_order_value

Business calculations

gross_sales = SUM(quantity × unit_price)

net_sales = gross_sales - discount_amount - returns

Business questions

What was today's revenue?

What is the average order value?

Which day generated the highest revenue?

What is the cancellation rate?

26. Gold — Customer 360

Table

gold.customer_360

Required attributes:

customer_id

customer_name

first_order_date

last_order_date

total_orders

total_quantity

total_spend

average_order_value

days_since_last_order

preferred_category

preferred_device

customer_segment

Customer segmentation

Example:

VIP

HIGH_VALUE

LOYAL

ACTIVE

INACTIVE

Participants must define and document their segmentation rules.

27. Gold — Product Performance

Table

gold.product_performance

Metrics:

product_id

product_name

category

units_sold

total_orders

gross_revenue

discount_amount

net_revenue

average_selling_price

unique_customers

Participants must calculate:

revenue_rank

category_rank

using Spark window functions.

Business question

What are the top 3 products by revenue within every category?

28. Gold — Customer Behavior

Table

gold.customer_behavior

Combine:

Customers

+

Clickstream

+

Orders

Metrics:

sessions

product_views

add_to_cart_events

checkout_events

purchases

Calculate:

view_to_cart_rate

cart_to_checkout_rate

checkout_to_purchase_rate

overall_conversion_rate

29. Gold — Product Conversion

Table

gold.product_conversion

Metrics:

product_id

product_views

add_to_cart_count

checkout_count

purchase_count

view_to_cart_rate

cart_to_purchase_rate

overall_conversion_rate

Business question

Which products have high customer interest but low purchase conversion?

30. Gold — Inventory Sales

Table

gold.inventory_sales

Combine:

Sales

+

Inventory

+

Products

Metrics:

product_id

sales_quantity

available_inventory

reserved_inventory

inventory_status

Identify:

OUT_OF_STOCK

LOW_STOCK

HEALTHY_STOCK

Business questions

Which high-selling products are out of stock?

Which products need replenishment?

Which products have excess inventory?

Advanced Business Challenges

These challenges are optional for advanced participants.

Challenge A — RFM Analysis

Calculate:

Recency

Days since last purchase.

Frequency

Number of orders.

Monetary

Total customer spend.

Classify customers:

CHAMPIONS

LOYAL

POTENTIAL_LOYALIST

AT_RISK

LOST

Challenge B — Abandoned Cart

Identify customers who:

PRODUCT_VIEW



ADD_TO_CART



CHECKOUT



NO PURCHASE

Create:

gold.abandoned_cart

Challenge C — Product Affinity

Identify products frequently purchased together.

Example:

Laptop + Mouse

Phone + Case

Laptop + Keyboard

Create:

gold.product_affinity

Challenge D — Customer Trend

Calculate:

customer_id

month

monthly_spend

previous_month_spend

growth_percentage

Classify customers as:

GROWING

STABLE

DECLINING

Phase 10 — Data Quality

Participants must implement data-quality checks.

Required checks

Customers

customer_id IS NOT NULL

email IS NOT NULL where required

Orders

order_id IS NOT NULL

customer_id IS NOT NULL

total_amount >= 0

OrderItems

quantity > 0

unit_price >= 0

discount BETWEEN 0 AND 1

Products

product_id IS NOT NULL

price >= 0

Payments

payment_id IS NOT NULL

payment_amount > 0

Referential integrity

Identify:

Orders without Customers

OrderItems without Orders

OrderItems without Products

Payments without Orders

The original project explicitly requires data cleaning, consistency validation and data-quality checks.

-

Phase 11 — Lakeflow Declarative Pipelines / DLT

Participants must implement the Silver and Gold transformation pipelines using the required Databricks pipeline technology.

Pipeline structure:

BRONZE

|



Silver Customers

Silver Orders

Silver OrderItems

Silver Payments

Silver Products

Silver Inventory

Silver Clickstream

|



GOLD

|

├— Daily Sales

├— Customer 360

├— Product Performance

├— Customer Behavior

├— Product Conversion

└— Inventory Sales

The original capstone specifies DLT as mandatory for transformations.

Participants must demonstrate:

Pipeline configuration

Data-quality expectations

Transformation dependencies

Successful pipeline execution

Handling of invalid records

Phase 12 — Unity Catalog Governance

Create the following structure:

ecommerce

|

|— bronze

|— silver

|— gold

Tables should be organized according to the architecture.

The original project requires Unity Catalog registration, access control, schema organization and lineage.

Access Control

Create groups such as:

data_engineers

data_analysts

business_users

Suggested access model:

Object	Data Engineer	Analyst	Business User
Bronze	Full	No	No
Silver	Full	Read	No
Gold	Full	Read	Read

Participants must implement and demonstrate the permissions.

36. Sensitive Data

Customer data contains attributes such as:

email

phone

shipping_address

Participants must identify sensitive attributes and decide how they should be governed.

Business users should consume business-ready Gold datasets rather than receiving unrestricted access to raw customer information.

Data Lineage

Participants must demonstrate lineage.

Example:

SQL Server Orders



Bronze Orders



Silver Orders



Gold Daily Sales

Another example:

ADLS Clickstream



Bronze Clickstream



Silver Clickstream

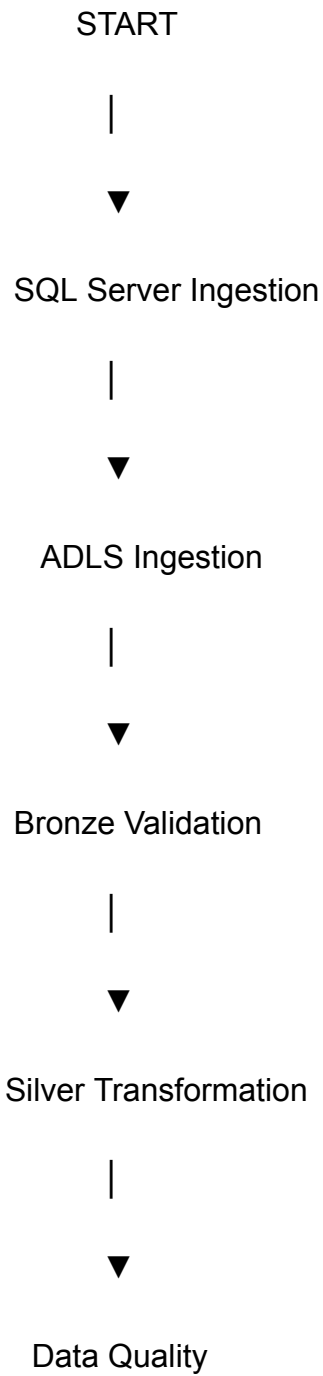


Gold Customer Behavior

Participants must explain how a business metric can be traced back to its source.

Phase 14 — Lakeflow Jobs / Workflow

Create an end-to-end workflow.



|



Gold Transformation

|



Business Validation

|



Notification

|



END

Participants must configure:

Task dependencies

Execution order

Retry behavior

Failure handling

Monitoring

The original project requires workflow dependencies, execution order and retry mechanisms.

Phase 15 — Testing

Participants must test the pipeline using multiple scenarios.

Test 1 — Initial Load

Verify all initial data reaches Bronze.

Test 2 — Incremental Load

Add new files/records.

Verify only the incremental data is processed.

Test 3 — Duplicate Data

Introduce duplicate records.

Verify Silver removes or handles them.

Test 4 — Invalid Data

Introduce:

negative amount

invalid quantity

invalid discount

NULL foreign key

Verify the defined quality strategy.

Test 5 — Update

Modify an existing customer/order/product.

Verify the latest information is represented correctly.

Test 6 — Referential Integrity

Introduce an order with an unknown customer.

Verify the expected quality behavior.

Final Business Questions

The completed platform must answer at least:

Sales

What is daily revenue?

What is monthly revenue?

What is average order value?

What is the cancellation rate?

What is the return rate?

Customers

Who are the top 10 customers?

What is each customer's lifetime spend?

Which customers are inactive?

Which customers are high-value?

Which categories does each customer prefer?

Products

What are the top products by revenue?

What are the top products by quantity?

What are the top products within each category?

Which products have low conversion?

Customer Behavior

How many product views occur each day?

What is the add-to-cart rate?

What is the checkout conversion rate?

What is the purchase conversion rate?

Inventory

Which products are out of stock?

Which high-selling products have low inventory?

Which products need replenishment?